GUIA TÉCNICO

Modelagem relacional e normalização

Chaves, dependências, integridade referencial e evolução do esquema

Propósito — Material de referência complementar aos notebooks. Exemplos locais com dados sintéticos; laboratórios externos somente sob configuração explícita.

Como utilizar — Leia as explicações, execute os exemplos no ambiente indicado e consulte os notebooks correspondentes.

ROTEIRO DO DOCUMENTO

1. 02 | Modelagem relacional e normalização
2. Objetivo e escopo
3. Conceito central: Dependências funcionais
4. Aplicação, ressalvas e limites
5. Entidades e cardinalidade
6. Normalização de tabelas
7. Integridade referencial - experimento
8. Exercício resolvido - identificar anomalia

Modelagem relacional e normalização

Chaves, dependências, integridade referencial e evolução do esquema

MAPA DE ESTUDO

01 Entidades, atributos e relacionamentos

02 Chaves e integridade referencial

03 Normalização e dependências funcionais

04 Condições de junção e cardinalidade

05 Modelagem orientada às consultas

Material de referência complementar aos notebooks. Exemplos locais com dados sintéticos; laboratórios externos somente sob configuração explícita.

02 | Modelagem relacional e normalização

GUIA TÉCNICO APROFUNDADO • ANÁLISE, CÓDIGO E INTERPRETAÇÃO

Objetivo e escopo

Estudo independente baseado no notebook aprovado. Os exemplos locais usam SQLite e dados sintéticos; PostgreSQL, MongoDB e Cassandra requerem serviços de laboratório para verificação real.

Conceito central: Dependências funcionais

A primeira forma normal evita campos multivalorados; a segunda remove dependências parciais de uma chave composta; a terceira separa dependências transitivas de atributos não-chave. A normalização reduz anomalias, porém desnormalização controlada pode atender necessidades de leitura.

Aplicação, ressalvas e limites

Teste o esquema tentando inserir uma leitura com sensor inexistente. Essa falha é evidência concreta de integridade referencial quando PRAGMA foreign_keys está habilitado.

Entidades e cardinalidade

No caso IoT, cômodos possuem vários sensores e sensores geram várias leituras (relações 1:N). A chave estrangeira de sensores aponta para cômodos; leituras aponta para sensores.

Exemplo do notebook — execução local

```
import sqlite3
conn = sqlite3.connect(':memory:')
conn.execute('PRAGMA foreign_keys=ON')
conn.executescript((Path('sql')/'01_schema_sqlite.sql').read_text(encoding='
    utf8'))
leituras = pd.read_csv(DATA/'leituras_sensores.csv')
leituras.to_sql('leituras',conn,if_exists='append',index=False)
print('SQLite inicializado:',conn.execute('SELECT COUNT(*) FROM leituras').f
    etchone()[0], 'leituras')
```

Normalização de tabelas

1FN: valores atômicos e sem listas em colunas. 2FN: sem dependência parcial de chave composta. 3FN: sem dependência transitiva entre atributos não chave. Normalização previne anomalias; desnormalização deve responder a uma necessidade medida.

Exemplo do notebook — execução local

```
q='''SELECT c.nome AS comodo,s.id_sensor,l.id_evento,l.temperatura_c FROM co
    modos c JOIN sensores s ON c.id_comodo=s.id_comodo JOIN leituras l ON s.
    id_sensor=l.id_sensor LIMIT 8''';display(pd.read_sql_query(q,conn))
```

Integridade referencial - experimento

Tentaremos inserir uma leitura vinculada a sensor inexistente. Em um esquema correto, a operação falha e não grava a linha.

Exemplo do notebook — execução local

```
try:
 conn.execute("INSERT INTO leituras VALUES (9999,999,'2025-01-01',21,50)")
except sqlite3.IntegrityError as exc:
 print('Restrição funcionou:',exc)
else:
 raise AssertionError('A inserção inválida não deveria ter sido permitida')
```

Exercício resolvido - identificar anomalia

Se o nome do cômodo aparecesse repetido em toda leitura, sua mudança exigiria atualizar várias linhas. Separar comodos evita inconsistência. Verificação: a tabela de leituras guarda somente id_sensor.

Exemplo do notebook — execução local

```
print('Colunas:',[r[1] for r in conn.execute('PRAGMA table_info(leituras)')]
    )
```

--- ### Aprofundamento v2 - Dependências funcionais e anomalias

Uma dependência funcional A -> B afirma que o mesmo valor de A determina B no domínio da relação; amostras finitas não comprovam uma regra de negócio. Revise 1FN, 2FN, 3FN e BCNF segundo a semântica das dependências, distinguindo desnormalização deliberada de modelagem acidental.

Interpretação: compare o resultado com a teoria anterior; este exemplo local não substitui um laboratório de serviço externo.

Exemplo do notebook — execução local

```
import sqlite3
with sqlite3.connect(':memory:') as db:
    db.execute('PRAGMA foreign_keys=ON')
    db.execute('CREATE TABLE clientes(id INTEGER PRIMARY KEY, email TEXT NOT
     NULL UNIQUE)')
    db.execute('CREATE TABLE compras(id INTEGER PRIMARY KEY, cliente_id INTE
    GER REFERENCES clientes(id))')
    db.execute("INSERT INTO clientes VALUES (1, 'ana@exemplo.local')")
    db.execute('INSERT INTO compras VALUES (100, 1)')
    try:
        db.execute("INSERT INTO clientes VALUES (2, 'ana@exemplo.local')")
    except sqlite3.IntegrityError:
        print('Unicidade verificada')
    else:
        raise AssertionError('UNIQUE não aplicado')
```

Exercício de revisão

Exercício resolvido - revisão: identifique a principal hipótese do exemplo acima, explique por que ela importa e cite uma condição na qual a técnica falharia. Resposta orientadora: registre a pré-condição (tipo de dado, chave, restrição, volume ou topologia), teste-a explicitamente e explique os limites dos resultados observados.

Aprofundamento: decisões e estudo de caso

Do problema à estrutura

Desenhe primeiro as entidades cômodo, sensor e leitura. Um cômodo pode possuir vários sensores e cada sensor, várias leituras. Colocar o nome do cômodo em cada leitura cria redundância: a atualização do nome precisaria

alcançar todas as observações, com risco de divergências. A decomposição em entidades ligadas por FK transforma essa alteração em uma única operação lógica.

Normalização com hipóteses

A análise de 2FN depende de identificar corretamente as chaves candidatas. Se a chave da tabela for simples, dependências parciais não ocorrem em relação a ela. Em 3FN, procure atributos não-chave determinados por outros atributos não-chave. Não se trata de quebrar todas as tabelas em pequenos fragmentos; normalização exige representar dependências reais do domínio.

Ação de exclusão e integridade

Considere a remoção de um sensor com histórico de leituras. ON DELETE CASCADE apagaria registros relacionados; RESTRICT impediria a remoção; SET NULL exigiria que o campo aceitasse ausência. Essas escolhas são regras de negócio. O laboratório testa rejeição de FK inválida, mas não resolve sozinho qual política de retenção ou anonimização seria adequada.

Exercício de modelagem

Proponha um esquema para cômodos, sensores, leituras e calibrações. Escreva PKs, FKs e restrições de domínio. Em seguida, descreva uma anomalia de inserção, uma de atualização e uma de exclusão de uma versão não normalizada. Demonstre no SQLite pelo menos uma violação deliberada, com tratamento explícito de IntegrityError.

Checklist de interpretação

• Defina a pergunta, unidade de análise e pré-condições da consulta.

• Registre chaves, cardinalidade, restrições e tratamento de valores ausentes.

• Execute os exemplos locais antes de habilitar serviços reais.

• Separe evidência observada de hipóteses e propriedades não testadas.

Leituras e arquivos de apoio

Consulte o notebook correspondente, os scripts em sql/, os datasets sintéticos e a documentação oficial dos SGBDs. Os resultados locais não substituem testes de integração com serviços externos.